Priority-Based Task Scheduling in a Computer System:
A First-Come, First-Served (FIFO) Tie-Breaking Approach

[Student Name]

[Team Members: Unsorted List, Sorted List, Binary Heap]

[Department, Institution Name]

[Course Code and Course Name]

[Instructor Name]

[Submission Date]

Priority-Based Task Scheduling in a Computer System: A FIFO Tie-Breaking Approach

1. Problem Statement

A computer system runs many tasks at the same time, and not all of them are equally important. A security alert must be handled before a disk cleanup, and an antivirus scan is more urgent than a document sent to the printer. The operating system therefore needs a scheduler that always chooses the most important waiting task, even while new tasks keep arriving.

The assignment asks for a priority-based task scheduler that can (a) insert at least six tasks, (b) select and remove the next task to run, (c) react correctly when a new high-priority task arrives, (d) define how ties between tasks of equal priority are resolved, (e) implement the scheduling logic, and (f) compare the time complexity of an unsorted list, a sorted list, and a binary heap. The project was divided among four team approaches: the unsorted list, the sorted list, the binary heap (priority queue), and the FIFO tie-breaking approach. This report covers the fourth approach, in which equal-priority tasks are served in the order in which they arrived.

2. Introduction

Scheduling is one of the core duties of an operating system. When several processes are ready to run, the scheduler decides which one receives the processor next (Silberschatz et al., 2018). In priority scheduling each process carries a priority value and the process with the highest priority is dispatched first. Priority rules are also the basis of real-time scheduling, where urgent work must meet its deadlines (Liu & Layland, 1973; Tanenbaum & Bos, 2015).

The natural data structure for this job is the priority queue, an abstract data type that supports inserting an item and removing the item with the highest priority (Cormen et al., 2022). A priority queue can be built on an unsorted list, a sorted list, or a binary heap, the structure introduced by Williams (1964) for heapsort. Each choice gives a different balance between the cost of inserting and the cost of selecting.

A priority queue alone leaves one question open: what happens when two tasks have exactly the same priority? If the rule is not defined, the order of those tasks depends on internal details of the data structure, and a task that arrived early can be overtaken by one that arrived later. The most natural fair rule is first-come, first-served (FIFO): among equal priorities, the task that has waited longest runs first. This report designs, implements, visualizes, and analyzes a scheduler that combines priority selection with FIFO tie-breaking.

3. Objectives

The objectives of this work are as follows:

- to build a working priority scheduler in Python that always selects the highest-priority task;

- to define and enforce a clear tie-breaking rule, namely first-come, first-served among equal priorities;
- to demonstrate inserting six or more tasks, selecting and removing the next task, and handling a late high-priority arrival;
- to visualize the queue after each event so that the behavior can be checked by eye;
- to verify the implementation with automated tests; and
- to compare the time and space complexity of the unsorted list, sorted list, and binary heap approaches.

4. Explanation of the Problem Statement

Priority convention. Every task has an integer priority. In this project a smaller number means a higher priority, so priority 1 is the most urgent. This is the usual convention for min-heaps and matches the way many systems number their priority levels.

Operations. The scheduler must support three operations: *insert*, which adds a task at any time; *peek*, which shows the next task without removing it; and *extract_next*, which removes and returns the task that should run now. A new task can arrive at any moment, including after other tasks have already been removed, so the structure must stay correct without being rebuilt from scratch.

Ties. A tie occurs when two or more waiting tasks have the same priority. In the demonstration data, three tasks have priority 2 and two tasks have priority 3. Without a tie rule, the scheduler might run any of them first. A plain binary heap does not preserve insertion order among equal keys (Sedgewick & Wayne, 2011), so a rule must be added deliberately. The rule chosen here is that the task with the earlier arrival wins. This is fair, because no task can be overtaken by a later task of the same importance, and it is predictable, because the output for a given input is always the same.

5. Proposed Solution and Approach

The solution gives every task an *arrival number*, a counter that increases by one each time a task is inserted. Tasks are then ordered by the composite key (priority, arrival number), compared from left to right. Because arrival numbers are unique, no two tasks ever have the same key, so the order is a strict total order and every selection is deterministic. The same sequence-number technique is recommended in the Python documentation for heap-based priority queues (Python Software Foundation, n.d.).

The tasks are stored in a binary min-heap, so that the task with the smallest key is always at the root. Insertion appends the task at the end of the array and moves it up while it is smaller than its parent. Removal takes the root, moves the last task to the root, and moves it down while a child is smaller. Both operations take O(log n) time. The heap is written by hand, instead of using the built-in *heapq* module, so that the data structure concepts are visible in the code.

The scheduler has a switch, *use_fifo*. When it is on, the key is (priority, arrival number). When it is off, the key is the priority alone. This allows an experiment that shows what goes wrong without the tie-break rule (Section 11). The project also includes a demonstration program, a chart generator, and a unit test file, and is organized for upload to a Git repository.

6. Real-World Applications

Priority scheduling with a FIFO rule inside each priority level appears in many systems:

- **Operating systems.** Schedulers keep a queue per priority level. The POSIX real-time policy SCHED_FIFO runs processes of equal priority in first-come, first-served order (Silberschatz et al., 2018).
- **Print and job spoolers.** Urgent jobs go first, and jobs of the same urgency are printed in the order they were submitted.
- **Network routers.** Packets are classified into priority classes for quality of service, and packets within a class are forwarded in arrival order.
- **Hospital triage and help desks.** Critical cases or tickets are handled first, and equally urgent ones are handled in order of arrival.
- **Event-driven simulation.** Events are processed by time, and events with the same time are processed in the order in which they were created.

7. Data Structures Used

The implementation uses three related structures, summarized in Table 1. The *Task* record holds the task data. The *heap array* stores tasks in heap order using the standard index rules: the parent of index i is (i − 1) // 2, and its children are 2i + 1 and 2i + 2 (Goodrich et al., 2013). The heap has two properties: the shape property, which says that the tree is complete and so fits in an array without gaps, and the order property, which says that every task has a key no larger than the keys of its children (Cormen et al., 2022). The *counter* is a single integer that supplies the next arrival number.

Table 1
Components of the FIFO Tie-Breaking Scheduler

Component	Type	Purpose
name	string	Identifies the task.
priority	integer	Urgency of the task; 1 is the highest priority.
arrival_seq	integer	FIFO stamp assigned when the task is inserted; breaks ties.
burst_time	integer	Simulated running time, used by the visualization.
heap array	list of Task	Holds all waiting tasks in binary min-heap order.
counter	integer	Next arrival number to hand out; increases by 1 per insert.

8. Algorithm and Pseudocode

The comparison function, insertion, and removal are given below. Index positions start at 0.

```
FUNCTION Less(a, b):
    RETURN (a.priority, a.arrival_seq) < (b.priority, b.arrival_seq)

PROCEDURE Insert(name, priority):
    task <- new Task(name, priority, arrival_seq = counter)
    counter <- counter + 1
    append task to heap
    SiftUp(last index)

PROCEDURE SiftUp(i):
    WHILE i > 0:
        parent <- (i - 1) DIV 2
        IF Less(heap[i], heap[parent]):
            swap heap[i], heap[parent]; i <- parent
        ELSE: STOP

FUNCTION ExtractNext():
    IF heap is empty: RETURN none
    top <- heap[0]
    last <- remove last element of heap
    IF heap is not empty: heap[0] <- last; SiftDown(0)
    RETURN top

PROCEDURE SiftDown(i):
    LOOP:
        smallest <- i; left <- 2i + 1; right <- 2i + 2
        IF left < size AND Less(heap[left], heap[smallest]):
            smallest <- left
        IF right < size AND Less(heap[right], heap[smallest]):
            smallest <- right
        IF smallest = i: STOP
        swap heap[i], heap[smallest]; i <- smallest
```

The only difference from an ordinary priority heap is the second element of the key in *Less*. That one change turns an unordered tie into a first-come, first-served tie.

9. Step-by-Step Demonstration

The demonstration program inserts six tasks: Backup Files (priority 3), Antivirus Scan (2), Disk Cleanup (4), Email Sync (2), Software Update (3), and Print Document (2). Table 2 shows the internal heap array after each operation, and Figure 1 shows the resulting queue order.

Table 2

Heap Array After Each Operation in the Demonstration

Step	Operation	Heap array, index 0 first (name, priority, arrival number)
1	Insert Backup Files (p3)	Backup (p3,#0)
2	Insert Antivirus Scan (p2)	Antivirus (p2,#1), Backup (p3,#0)

Step	Operation	Heap array, index 0 first (name, priority, arrival number)
3	Insert Disk Cleanup (p4)	Antivirus (p2,#1), Backup (p3,#0), Disk Cleanup (p4,#2)
4	Insert Email Sync (p2)	Antivirus (p2,#1), Email Sync (p2,#3), Disk Cleanup (p4,#2), Backup (p3,#0)
5	Insert Software Update (p3)	Antivirus (p2,#1), Email Sync (p2,#3), Disk Cleanup (p4,#2), Backup (p3,#0), Sw Update (p3,#4)
6	Insert Print Document (p2)	Antivirus (p2,#1), Email Sync (p2,#3), Print (p2,#5), Backup (p3,#0), Sw Update (p3,#4), Disk Cleanup (p4,#2)
7	Remove Antivirus Scan	Email Sync (p2,#3), Backup (p3,#0), Print (p2,#5), Disk Cleanup (p4,#2), Sw Update (p3,#4)
8	Insert Security Alert (p1)	Sec. Alert (p1,#6), Backup (p3,#0), Email Sync (p2,#3), Disk Cleanup (p4,#2), Sw Update (p3,#4), Print (p2,#5)
9	Insert Cloud Upload (p2)	Sec. Alert (p1,#6), Backup (p3,#0), Email Sync (p2,#3), Disk Cleanup (p4,#2), Sw Update (p3,#4), Print (p2,#5), Cloud Upload (p2,#7)

Note. Names are shortened. In each entry, p is the priority and # is the arrival number.

Steps 1–6: inserting six tasks. Each task is appended and moved up if it beats its parent. After step 6 the root is Antivirus Scan (priority 2, arrival 1). Three tasks share priority 2, and their order in the queue is arrival 1, 3, and 5, which is the FIFO order.

Step 7: selecting and removing the next task. Antivirus Scan is removed because it has the best key. Email Sync, the next oldest priority-2 task, moves to the root.

Step 8: a high-priority task arrives. Security Alert (priority 1, arrival 6) is inserted and moves up to the root, ahead of every waiting task. It will run next, even though it arrived last.

Step 9: a tie with a later arrival. Cloud Upload (priority 2, arrival 7) arrives. It does not overtake Email Sync (arrival 3) or Print Document (arrival 5), because its arrival number is larger. It sits behind them and ahead of the priority-3 and priority-4 tasks.

Draining the queue. The remaining tasks run in this order: Security Alert, Email Sync, Print Document, Cloud Upload, Backup Files, Software Update, and Disk Cleanup. Figure 2 shows the complete run order.

Figure 1

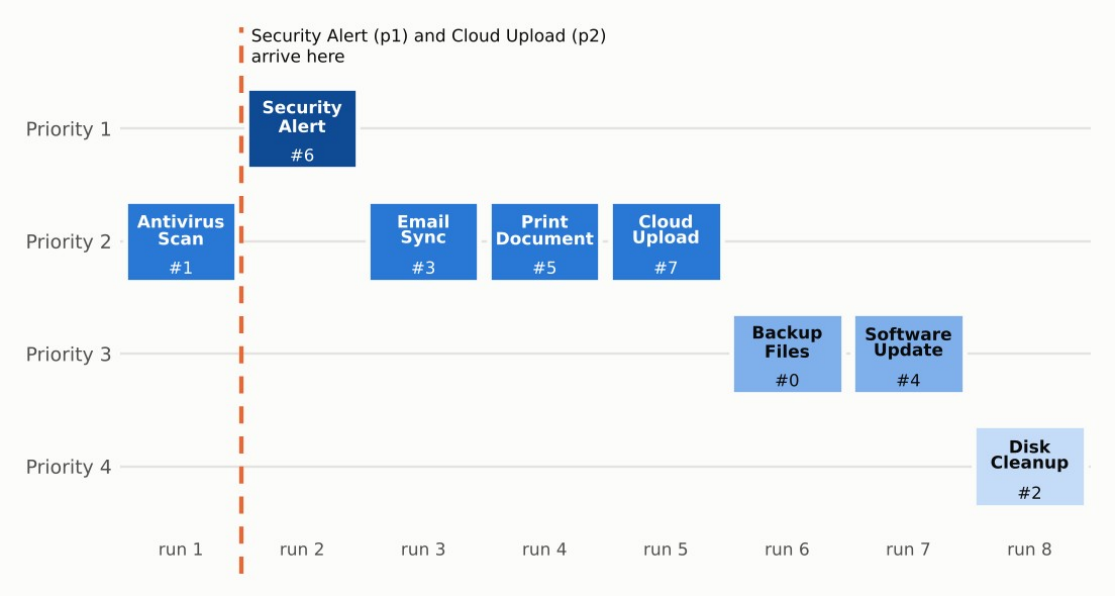
Queue Order After Each Event



Note. Rank 1 runs next. Darker blocks have higher priority. An orange outline marks the task that has just arrived.

Figure 2

Execution Timeline Grouped by Priority



Note. The number in each block is the arrival number. Tasks of equal priority run in arrival order.

10. Time and Space Complexity

Let n be the number of tasks waiting in the queue. Table 3 compares the three structures required by the assignment with the heap used in this approach (Cormen et al., 2022; Knuth, 1998).

Table 3

Complexity Comparison of the Priority Queue Implementations

Operation	Unsorted list	Sorted list	Binary heap	Heap with FIFO key (this work)
Insert	O(1)	O(n)	O(log n)	O(log n)
Peek next	O(n)	O(1)	O(1)	O(1)
Extract next	O(n)	O(1)*	O(log n)	O(log n)
n inserts and n extracts	$O(n^2)$	$O(n^2)$	O(n log n)	O(n log n)
Space	O(n)	O(n)	O(n)	O(n)
Equal priorities	Depends on scan rule	Depends on insert rule	Not defined	FIFO guaranteed

Note. *Assumes the list is kept so that the best task is at the end of the array and can be removed in constant time.

Unsorted list. Inserting appends at the end, which takes O(1). Selecting requires scanning every task to find the best one, which takes O(n). **Sorted list.** Selecting is O(1) because the best task is at a known end, but insertion must find the right place and shift later items, which takes O(n). **Binary heap.** The tree has height about $\log_2 n$, so insertion and removal each move a task along at most one root-to-leaf path, which takes O(log n). Reading the root takes O(1).

Effect of the FIFO key. Comparing two keys costs a constant amount of work, so using the tuple (priority, arrival number) does not change any of the asymptotic bounds. The counter adds O(1) per insertion and one integer of extra space overall. The space used is O(n) for the heap array, with one stored arrival number per task. For a workload of n tasks the heap approach needs O(n log n) in total, against $O(n^2)$ for the two list approaches, which is why heaps are preferred when the queue is large.

11. Result and Conclusion

Results

The program produced the expected order in every scenario. Highest-priority tasks always ran first, a late priority-1 task moved to the front immediately, and a late priority-2 task waited behind older priority-2 tasks. All six automated tests passed. They cover priority order, FIFO ties, a late high-priority arrival, a late equal-priority arrival, an empty queue, and a randomized check of 200 queues against a sorted reference.

The comparison experiment used seven tasks, A to G, that all had priority 2 and arrived in that order. With the FIFO key off, the heap returned A, G, F, E, D, C, B, so six of

the seven tasks were served out of arrival order. With the FIFO key on, the order was A, B, C, D, E, F, G (Figure 3). This confirms that the tie-break rule is necessary and that it works.

Figure 3

Run Order of Seven Equal-Priority Tasks With and Without the FIFO Key



Note. An orange outline marks a task that is not in its arrival position.

Limitations

The scheduler does not preempt a running task, so a new high-priority task waits until the current task is removed. A steady stream of high-priority tasks can also starve low-priority tasks, a known weakness of priority scheduling that is usually reduced by aging, where waiting tasks gradually gain priority (Silberschatz et al., 2018). Neither feature was part of the assignment.

Conclusion

Priority scheduling needs a tie rule to be fully predictable. Adding an arrival number to the priority gives a strict first-come, first-served order within each priority level, costs only constant extra work, and keeps the O(log n) running time of the binary heap. The implementation, charts, and tests show that the approach meets all requirements of the assignment. Together with the unsorted list and sorted list approaches developed by the team, it shows why a heap with a deterministic tie-break is the most practical choice for a scheduler that handles many tasks.

References

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms* (4th ed.). MIT Press.

Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2013). *Data structures and algorithms in Python*. Wiley.

Knuth, D. E. (1998). *The art of computer programming: Vol. 3. Sorting and searching* (2nd ed.). Addison-Wesley.

Liu, C. L., & Layland, J. W. (1973). Scheduling algorithms for multiprogramming in a hard-real-time environment. *Journal of the ACM, 20*(1), 46–61. https://doi.org/10.1145/321738.321743

Python Software Foundation. (n.d.). *heapq: Heap queue algorithm*. In *Python 3 documentation*. https://docs.python.org/3/library/heapq.html

Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4th ed.). Addison-Wesley.

Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating system concepts* (10th ed.). Wiley.

Tanenbaum, A. S., & Bos, H. (2015). *Modern operating systems* (4th ed.). Pearson.

Williams, J. W. J. (1964). Algorithm 232: Heapsort. *Communications of the ACM, 7*(6), 347–348. https://doi.org/10.1145/512274.512284